# Trusting the Formal Judge: Certifying LLM Theorem Scorers for Code Verification in Lean 4

**Anonymous Author(s)**
Affiliation
Address
email

## Abstract

Formal methods provide unparalleled rigor in software verification, and with the advent of AI systems, we are beginning to see their potential realized at scale; however, evaluating the quality of the theorems and specifications these systems generate remains a critical bottleneck. Traditional benchmarks often provide a fixed theorem statement, but agentic systems must infer and synthesize these properties on their own, demanding a scalable and reliable evaluation method. We present an LLM-based judge designed to score the quality of generated Lean 4 theorems based on correctness, completeness, and semantic alignment with a reference implementation. More importantly, we introduce a novel methodology to certify the judge's trustworthiness by verifying its adherence to fundamental logical properties. We demonstrate that our judge satisfies identity (identical inputs yield maximal scores), monotonicity (scores decrease predictably as bugs are introduced or specifications are removed), and stability/invariance under prompt and rewrite perturbations (e.g., GPT-5.3-Codex: Spearman $\rho = -0.790$ on bug ladders over 176 files, minimum prompt-pair Spearman $\rho = 0.955$, Cronbach's $\alpha = 0.942$). We further calibrate the binary decision boundary via a threshold sweep and show that stricter cutoffs reduce repeat-call decision flips ($0.058 \to 0.046$, $t = 0.60 \to 0.85$) while preserving ranking behavior. By applying this validated judge to the VERIBENCH[29]scalable oversight benchmark, we show that agentic approaches for formal code generation significantly outperform baseline prompting, with a top score of 0.615 vs. 0.470. This work provides a blueprint for creating and validating reliable automated judges, enabling robust and scalable evaluation of next-generation theorem-proving agents.

## 1 Introduction

Agentic AI systems are emerging as a powerful tool for formal code verification, capable of not just proving theorems but also generating the code and the theorems themselves. This end-to-end synthesis presents a profound evaluation challenge. In traditional auto-formalization benchmarks, models are given a precise natural language statement to formalize. However, in a more realistic, agentic setting, the system must infer and propose relevant theorems from a source program and its docstring. This autonomy makes manual evaluation intractable and requires automated, scalable solutions [38, 11, 8].

We propose a radical approach: use LLMs themselves as judges of specification quality. But can you really trust such a judge? An unreliable judge can produce misleading results, rewarding stylistic mimicry over semantic correctness and hindering genuine progress. Without a principled validation method, LLM-based evaluation remains ad hoc and its results questionable [12, 13].

In this paper, we present evidence indicating that it is possible to trust an LLM judge. We introduce an LLM judge for scoring the quality of synthesized theorems in Lean 4 and, crucially, present a

Submitted to 39th Conference on Neural Information Processing Systems (NeurIPS 2025). Do not distribute.

methodology for validating its trustworthiness. We demonstrate that an LLM judge, when properly validated, can serve as a reliable proxy for human evaluation, enabling the scalable assessment of complex, agentic systems.

Our **contributions** are:

1. **An LLM Judge for Theorem Quality:** We design an LLM judge that scores generated Lean 4 artifacts on a 0–10 scale based on correctness, completeness, and semantic alignment.
2. **A Trustworthiness Validation Methodology:** We propose and execute a set of property checks to certify the judge's reliability and trustworthiness, verifying its adherence to logical properties like identity, monotonicity, and prompt-level stability/invariance under controlled perturbations, then calibrating the pass/fail threshold to minimize decision instability.
3. **Empirical Demonstration of the Judge on VERIBENCH:** We apply our validated judge to evaluate multiple agentic frameworks, providing evidence that feedback-driven methods significantly outperform non-agentic baselines in theorem-synthesis quality.

## 2 Related Work

**LLM-as-a-Judge.** LLMs are increasingly used as *automated evaluators* across open-ended tasks, often reaching strong agreement with human ratings—e.g., GPT-4 judges achieving $>80\%$ agreement with human preference labels in dialogue [36]. However, large-scale audits caution that reliability varies by task and setup, and stress the need for explicit validation before replacing human judgment [12, 13]. Common biases include position and verbosity effects and sensitivity to prompt phrasing [36, 13].

**Improving judge reliability.** To mitigate biases and instability, multi-agent debate frameworks such as *ChatEval* emulate panel-style review to yield more robust consensus scores [5]. Structured prompting like *G-Eval* elicits rubric-driven, stepwise reasoning prior to scoring, improving agreement with human criteria on NLG tasks [15]. Reliability can also be quantified via repeated scorings and internal consistency measures (e.g., Cronbach's $\alpha$) to detect prompt/run sensitivity [13].

**Code and formal-spec evaluation.** For programming, single-pass metrics like pass@k can be brittle or unavailable; early LLM-judge approaches such as *ICE-Score* highlighted difficulty distinguishing partial vs. full correctness from a one-shot review [39]. *CodeJudge* operationalizes a two-stage "analyze-then-summarize" procedure with a taxonomy of code faults, significantly improving correlation with functional ground truth [28]. In formal verification/autoformalization, prior work typically provides fixed target theorems, reducing evaluation to proof success. In contrast, agentic systems must propose specifications, necessitating *semantic* judges. We adopt a certification protocol (akin to metamorphic testing) and verify that our LLM judge obeys *identity/consistency*, *monotonicity* with respect to injected bugs and specification omissions, and *prompt/rewrite stability*, directly answering recent calls for validated, trustworthy LLM evaluators in safety-critical domains [12, 13].

**Overview of VERIBENCH.** Concretely, VERIBENCH consists of 857 theorems divided into five subsets:

1. **HumanEval** – 387 theorems derived from 56 standard programming puzzles from Chen et al. [7];
2. **EasySet** – 278 theorems extracted from 41 introductory logic and programming tasks;
3. **CSSet** – 68 theorems drawn from 10 classical data-structure and algorithm problems;
4. **SecuritySet** – 112 theorems taken from 28 examples of buffer overflows, privilege escalation, and race-condition labs based on real code;
5. **RealCodeSet** – 12 theorems extracted from 5 Python standard library programs, used to test model performance on production-grade code.

This design covers a spectrum of tasks, from simple correctness theorems to more complex invariants, algorithmic properties, and real-world code verification. See the ICML AI4Math workshop poster for VERIBENCH for additional context [29].

**Evaluation Consistency** Recent work has shown that unconstrained chain-of-thought reasoning can reduce evaluation consistency, and that simple comparative prompting often outperforms complex multi-step pipelines [6]. We interpret these findings as evidence that stability, rather than reasoning complexity, is the primary driver of reliable grading. Our work formalizes this intuition by introducing certifiable structural properties—identity, monotonicity, and reliability—that predict agreement with human judgments.

# 3 Methodology: A Certification Protocol for LLM Judges

**Why is an LLM judge needed for theorem-quality scoring?** Evaluating AI-generated theorems/specifications requires a method to assess their quality, as proof assistants can verify a proof's correctness but not whether the agent proved the correct theorem. This is challenging because proving the logical equivalence of two arbitrary theorem statements is generally undecidable, and an agent may generate theorems that are semantically valid but syntactically different or incomparable with the gold reference specification/theorem (e.g., by splitting a theorem into different parts).

Our goal is to certify the trustworthiness of an LLM judge for the task of formal specification/theorem generation. This is akin to scoring the quality of a conjecture. We operationalize this by requiring the judge to compare a candidate Lean 4 artifact (implementation, tests, theorems) against its gold-standard Lean 4 reference. We provide the full set of Lean 4 artifacts because theorems might not make sense in isolation. The judge returns a discrete score $s \in \{0, 1, \ldots, 10\}$ for correctness, completeness, and semantic alignment, which we later normalize to $[0, 1]$. We use this score as a metric signal for evaluating the specifications generated by the agents.

To be considered trustworthy, the judge must satisfy the following falsifiable properties. These are necessary conditions for trust: they certify that the judge reacts logically to structured perturbations.

**P1 — Identity & Consistency.** The judge must be idempotent at the maximum score when comparing an artifact to itself. For any reference $X$, it must hold that $s(X, X) = 1$.

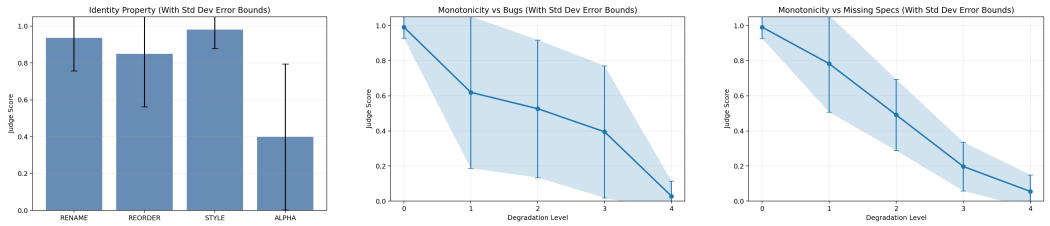
**P2 — Monotonicity to Bugs.** The judge's score must decrease as semantic bugs are cumulatively introduced. Let $B_1$ and $B_2$ be sets of bugs such that $B_1 \subset B_2$. For candidates $C_{B_1}$ and $C_{B_2}$ containing precisely these bug sets, the expected scores must satisfy $\mathbb{E}[s(C_{B_1})] > \mathbb{E}[s(C_{B_2})]$.

**P3 — Monotonicity to Missing Specs.** The judge's score must decrease as specification elements are removed. Let $S_1$ and $S_2$ be sets of specifications (theorems and tests) such that $S_2 \subset S_1$. For candidates $C_{S_1}$ and $C_{S_2}$ containing precisely these specification sets, the expected scores must satisfy $\mathbb{E}[s(C_{S_1})] > \mathbb{E}[s(C_{S_2})]$.

**P4 — Prompt Stability & Semantic Invariance.** The judge should produce stable scores under repeated calls and preserve judgments under semantics-preserving rewrites. Concretely, for fixed artifact pairs and fixed judge instructions, repeated-call variance should remain low, prompt-pair rank agreement should remain high under instruction-equivalent prompt rewordings, and style-level rewrites should induce near-zero score gaps. This property guards against prompt-fragility in deployment settings where wording and formatting vary.

**Certification Protocol.** We implement four scripted perturbation families on the *candidate* while holding the *gold* fixed: (i) *Identity* (P1), pairing each gold file with itself (and a semantically identical reflow) to test idempotence robustness; (ii) a *Bug Ladder* (P2), applying $k \in \{0, \ldots, K\}$ semantics-changing transformations (e.g., broken identities, violated commutativity, incorrect base cases); and (iii) *Specification Ablation* (P3), removing $m \in \{0, \ldots, M\}$ theorems or tests; and (iv) *Prompt Stability/Invariance* (P4), evaluating each pair under multiple instruction-equivalent prompt templates and repeated calls, then measuring repeatability, prompt agreement, and invariance gaps by variant type.[1] We then verify monotonicity by computing rank correlations (Kendall's $\tau$, Spearman's $\rho$) between $(k, m)$ and judged scores, and by reporting pairwise ordering accuracy (AUC) that "worse" artifacts receive lower scores [28, 39].

[1] We distinguish *prompt wording* from *task instruction*. P4 varies phrasing/format while preserving the same judging instruction and rubric. We do not include adversarial or instruction-changing prompts (e.g., "ignore the input") in this stability metric.




(a) Identity (Reflexivity) (b) Monotonicity vs. Bugs (c) Monotonicity vs. Missing Specs

Figure 1: **LLM Judge Trustworthiness Sanity Checks.** VeriBench-wide GPT-5.3-Codex API run ($n_{\text{files}} = 184$, $n_{\text{repeats}} = 3$, with standard-error overlays). **(a) Identity:** 736 syntactic-variant pairs yield mean score $0.792$ ($\sigma = 0.352$); STYLE/RENAME remain high while aggressive ALPHA-renaming is harder. **(b) Monotonicity vs. Bugs:** 920 ladder points show a negative trend (Pearson $r = -0.687$, Spearman $\rho = -0.808$); full-ladder weak monotonicity is $52.2\%$. **(c) Monotonicity vs. Missing Specs:** 920 points show stronger monotonic behavior (Pearson $r = -0.892$, Spearman $\rho = -0.899$); weak monotonicity is $89.1\%$.

**Decision-threshold calibration (Exp13).** After validating score-level behavior (P1–P4), we calibrate the binary operating point used for deployment decisions. For threshold $t \in [0, 1]$, we define PASS iff $\bar{s} \geq t$, where $\bar{s}$ is the mean over repeated calls for the same pair. We sweep $t \in \{0.60, 0.65, 0.70, 0.75, 0.80, 0.85\}$ and report: (i) *flip rate* (fraction of items whose repeat-level pass/fail decisions disagree), (ii) *pass rate* (coverage), and (iii) ranking diagnostics (Spearman bug-correlation) to verify that calibration affects classification stability, not score ordering. All reported threshold metrics include 95% bootstrap confidence intervals over files.

**Establishing trust in the LLM judge.** To ensure trustworthiness of the LLM judge in evaluating agentic autoformalization, we systematically validate key properties the judge must satisfy in Figure 1: (1) scores decrease consistently as more theorems and tests are omitted, (2) scores similarly degrade when more implementation bugs are introduced, and (3) semantically preserving edits keep scores high. In our latest GPT-5.3-Codex API sanity run ($n_{\text{files}} = 184$, $n_{\text{repeats}} = 3$), monotonic trends remain strong (bugs: Spearman $\rho = -0.808$, Pearson $r = -0.687$; missing specs: Spearman $\rho = -0.899$, Pearson $r = -0.892$), supporting use of the judge as a semantically meaningful proxy for correctness/completeness alignment across VERIBENCH subsets (Table 4) [13, 12].

# 4 Results: Evaluating Agentic Theorem Generation with a Certified Judge

Table 4 presents the normalized average scores across different splits of VERIBENCH: Easy Set (Easy), CS Set (CS), Real Python Code (Real), HumanEval Set (HE), and Security Set (Sec). As expected, the baseline prompting approach achieves the lowest overall normalized score (0.470), reflecting its limited capability in autonomously identifying comprehensive theorem properties. It also gives us further confidence in our judge. Conversely, agentic methods leveraging iterative self-debugging and self-improvement demonstrate significant enhancements. Specifically, the DSPy ReAct agent, which incorporates feedback-driven self-debugging, achieves the highest overall average score of 0.615. The Trace++ agent, employing both self-debugging and self-improvement mechanisms, also shows notable improvement, attaining an average score of 0.588. These results underscore the effectiveness of agentic, feedback-driven approaches in enhancing theorem-generation quality compared to non-agentic baselines [11, 8, 7].

## 4.1 New Codex API Sanity Run (April 18, 2026)

We added a new certification run in `codex_api_gpt53_r3_20260418_230027` under the experiment results directory. This run uses GPT-5.3-Codex in API judge mode with three repeated judge calls per pair ($n_{\text{repeats}} = 3$), over 184 Lean files (736 identity rows and 920 rows each for bug/missing ladders).

**What the data shows.** Monotonicity with missing specifications is strong ($\rho = -0.899$, $r = -0.892$, weak monotonicity $89.1\%$), indicating stable penalty as theorem content is removed. Bug monotonicity is directionally correct but noisier ($\rho = -0.808$, $r = -0.687$, weak monotonicity $52.2\%$), suggesting some bug ladders induce local score reversals. Identity is high for mild semantic-

| Identity Variant | Mean Score | Mean Within-Item Std |
|---|---|---|
| RENAME | 0.936 | 0.005 |
| REORDER | 0.849 | 0.029 |
| STYLE | 0.982 | 0.001 |
| ALPHA | 0.399 | 0.096 |

Table 1: **Identity breakdown for the Codex API run.** The low overall identity mean (0.792) is primarily driven by aggressive alpha-renaming.

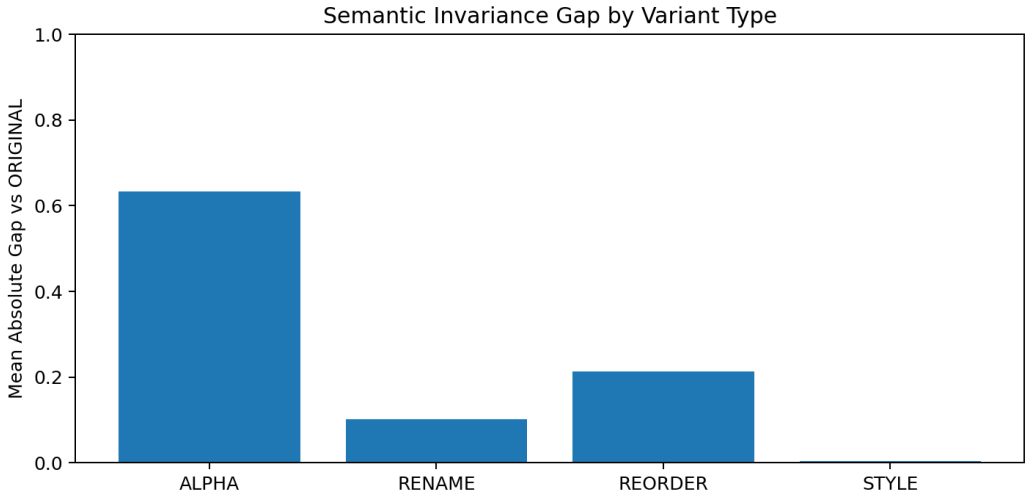



Figure 2: **P4 semantic invariance gaps by variant type.** Mean absolute score gap versus ORIGINAL across semantics-preserving perturbations in the dedicated stability run. The judge is highly invariant to STYLE, moderately sensitive to RENAME/REORDER, and most sensitive to ALPHA variants.

preserving edits but not uniform across all transforms, which is critical for interpreting aggregate identity means.

**Interpretation.** The judge reliably captures coarse semantic degradation (especially missing-spec ablations), while lexical/format-level robustness is transformation-dependent. Practically, this motivates canonicalization of binder names and stronger structure-aware prompting before using identity as a strict pass/fail criterion. The run metadata also notes appended datapoints from 34 added tasks scored in local mode, so direct cross-run comparisons should be treated as provisional until fully re-run under one scoring mode.

### 4.2 Stability and Semantic Invariance (April 20, 2026)

We add a dedicated P4 run in `stability_gpt-53-codex_full_20260420_185401` using GPT-5.3-Codex in API mode over 184 files, 4 prompt templates (`P1_BASELINE`, `P2_FORMAL`, `P4_CHECKLIST`, `P8_STRICT_B`), and 5 repeats per prompt ($n_{\text{cells}} = 3680$). With no prompt changes (same template, repeated calls), run-to-run noise is low (mean_repeat_std $= 0.0330$). Under instruction-equivalent prompt rewordings, cross-prompt score dispersion is moderate (0.0782), while overall internal consistency remains high (Cronbach's $\alpha = 0.9416$). Prompt pairs preserve ranking strongly (minimum Spearman $\rho = 0.9550$), but binary agreement is weaker near threshold (flip rate $= 31.3\%$, minimum Cohen's $\kappa = 0.4540$). Semantic invariance gaps are smallest for STYLE (0.0042), moderate for RENAME (0.1021) and REORDER (0.2128), and largest for ALPHA (0.6325), indicating that alpha-conversion handling is the dominant robustness bottleneck.

### 4.3 Threshold Calibration Sweep (Exp13, 176 files)

To convert continuous judge scores into a deployment-ready binary decision, we ran Exp13 on all 176 files from `veribench_dataset_newest` using GPT-5.3-Codex (`P8_STRICT_B`, $n_{\text{repeats}} = 3$, 1000 bootstrap resamples). The key finding is that thresholding primarily changes *decision stability*

| Threshold | Flip Rate (95% CI) | Pass Rate (95% CI) |
|---|---|---|
| 0.60 | 0.058 [0.048, 0.069] | 0.583 [0.559, 0.608] |
| 0.70 | 0.049 [0.040, 0.059] | 0.559 [0.537, 0.581] |
| 0.80 | 0.051 [0.042, 0.061] | 0.544 [0.523, 0.566] |
| 0.85 | **0.046** [**0.037**, **0.057**] | 0.536 [0.515, 0.557] |

Table 2: **Threshold calibration summary (Exp13, full 176-file run).** Calibration yields a modest but consistent reduction in decision flips at stricter thresholds, with moderate coverage loss. Ranking behavior is unchanged across thresholds ($\rho_{\text{bug}} = -0.790$ at all $t$).

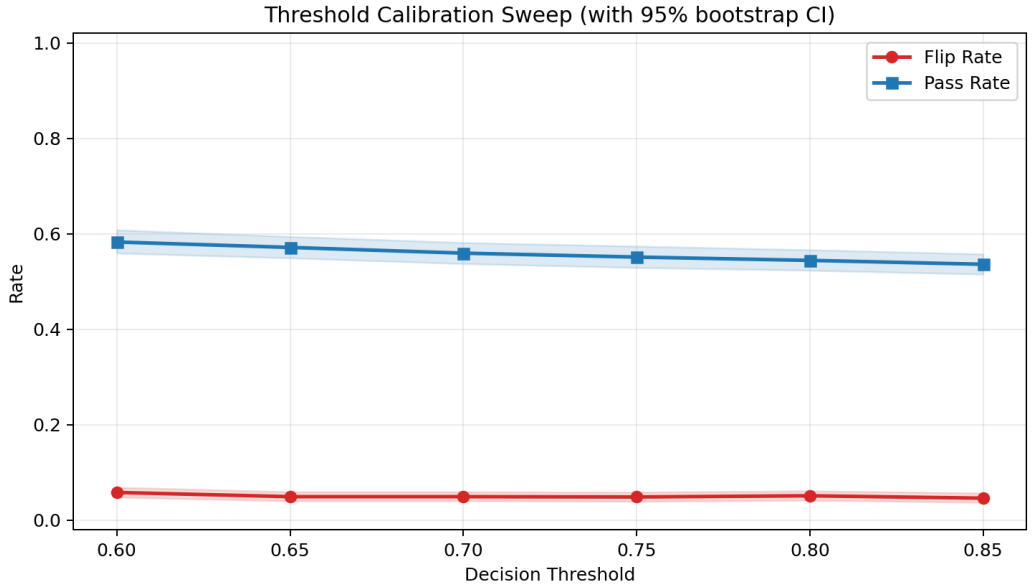



Figure 3: **Decision calibration curve from Exp13.** As threshold increases, pass rate declines smoothly; flip rate improves modestly and reaches its minimum at $t = 0.85$ in this run. This supports a calibration-first framing: improve binary reliability via threshold selection while preserving score-order semantics.

*and coverage*, not ranking behavior: across $t \in [0.60, 0.85]$, bug-ladder Spearman remains constant ($\rho = -0.790$, 95% CI $[-0.810, -0.769]$), while flip rate and pass rate move in opposite directions. Specifically, raising $t$ from 0.60 to 0.85 reduces flip rate from 0.058 to 0.046, while pass rate decreases from 0.583 to 0.536. Under our operating-point rule (minimize flip rate, tie-break by pass rate proximity to 0.5), $t = 0.85$ is selected.

### 4.4 Human Alignment at Final Robust Settings (Exp16/Exp17)

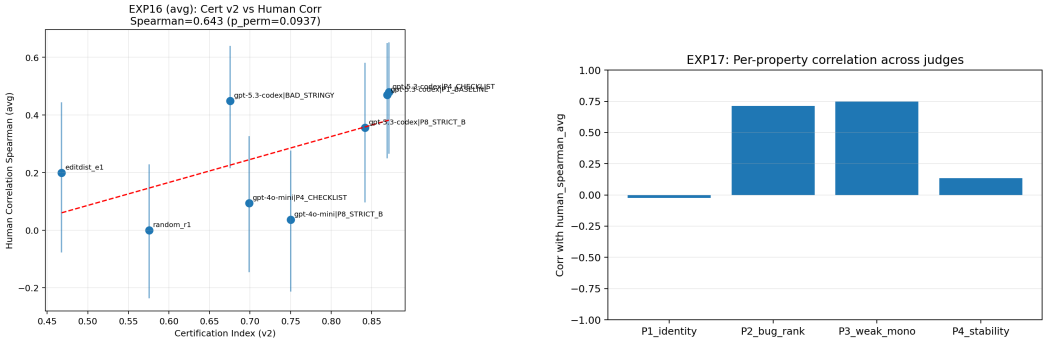
We ran a final multi-judge human-alignment suite over 72 aligned candidate–gold pairs (24 problems $\times$ 3 candidates; three pairs skipped due missing gold), using 8 judges and 3 human label sets: pass1, pass2, and their average. Judge-side uncertainty used 10,000 bootstrap iterations and 20,000 permutation iterations, with 3 repeated judge calls per pair and threshold $t = 0.85$.

**Experiment design.** **Exp16** asks whether certification quality predicts human agreement *across judges*: for each judge, compute a certification composite and correlate it with that judge's human Spearman correlation. **Exp17** decomposes this relation by property (P1–P4), and reports per-property correlations, partial correlations, and multivariate regression. We report both certification composites: **v1** (uses bug-rank and weak monotonicity directly) and **v2** (adds anti-string and extra penalties for instability).

**Main result (judge-level trend).** Certification score **v1** strongly predicts human agreement across all three human label sets, while **v2** is weaker and generally non-significant at this sample size

| Label Set | Trend v1 | Trend v2 |
|---|---|---|
| pass1 | $\rho = 0.810,\ p = 0.0218$ | $\rho = 0.500,\ p = 0.2156$ |
| pass2 | $\rho = 0.810,\ p = 0.0214$ | $\rho = 0.714,\ p = 0.0561$ |
| avg | $\rho = 0.881,\ p = 0.0080$ | $\rho = 0.643,\ p = 0.0937$ |

Table 3: **Exp16 final robust trend: certification → human alignment (Spearman, permutation $p$).** Each row uses all 8 judges. v1 is consistently stronger than v2 under the final robust setup.




(a) Exp16 (avg labels): certification score vs. human correlation.

(b) Exp17 (avg labels): per-property correlation with human agreement.

Figure 4: **Final robust human-alignment results.** Left: judge-level scaling trend (Exp16) using averaged human labels. Right: property-level decomposition (Exp17), showing P3 (weak monotonicity) and P2 (bug-rank monotonicity) as the strongest positive predictors of human alignment.

(Table 3). On averaged human labels, v1 reaches Spearman $\rho = 0.881$ (perm. $p = 0.0080$); pass1 and pass2 are both $\rho = 0.810$ (perm. $p \approx 0.021$). This supports the central claim that judges satisfying certification properties better are more human-aligned.

**Judge-level effect sizes.** Under averaged human labels, top certified judges are GPT-5.3-Codex prompts: `P4_CHECKLIST` ($r_s = 0.480,\ p = 2.0 \times 10^{-5}$), `P1_BASELINE` ($r_s = 0.470,\ p = 3.1 \times 10^{-5}$), `P8_STRICT_B` ($r_s = 0.356,\ p = 0.0022$). Lower-certification baselines are markedly weaker: `gpt-4o-mini|P8_STRICT_B` ($r_s = 0.037,\ p = 0.760$), `gpt-4o-mini|P4_CHECKLIST` ($r_s = 0.093,\ p = 0.435$), `random` ($r_s \approx 0,\ p = 0.995$).

**Property-level interpretation (Exp17).** For averaged labels, per-property correlations identify **P3 weak monotonicity** as strongest (Spearman $0.747,\ p = 0.043$; Pearson $0.744,\ p = 0.048$), with **P2 bug-rank monotonicity** next (Spearman $0.714,\ p = 0.056$; Pearson $0.636,\ p = 0.037$). P1 identity and P4 stability are not strong standalone predictors at this $n_{\text{judges}} = 8$ scale. Across pass1/pass2/avg, P3 is consistently top-ranked, indicating that semantic monotone behavior under controlled degradations is the most stable predictor of human agreement in this study.

### 4.5 Cross-Model Validation: Claude Sonnet (Preliminary)

We replicate the certification protocol using Claude Sonnet 4 as the judge model on a preliminary subset of three VeriBench files (one each from CS, Easy, and HumanEval), with five degradation levels and three repeated calls per pair. Identity (P1) remains perfect ($\mu = 1.000$, $\sigma = 0.000$), confirming that self-comparison yields maximal scores regardless of judge model. Bug monotonicity (P2) shows a clear negative trend (Pearson $r = -0.717$, Spearman $\rho = -0.674$, weak monotonicity $66.7\%$), and specification ablation (P3) is directionally correct (Pearson $r = -0.560$, Spearman $\rho = -0.402$). Prompt stability (P4) yields a pooled within-item standard deviation of $0.088$, comparable to the Codex run. These preliminary cross-model results suggest that the certification properties are not specific to a single LLM family; a full-scale Claude run across all VeriBench files is planned.

| Agent | Eval | Split | | | | | Avg |
|---|---|---|---|---|---|---|---|
| | | Easy | CS | Real | HE | Sec | |
| Baseline Prompting | Normalized | 0.580 | 0.690 | 0.472 | 0.410 | 0.347 | **0.470** |
| DSPy ReAct | Normalized | 0.659 | 0.754 | 0.580 | 0.650 | 0.454 | **0.615** |
| Trace+ (Self-Debug) | Normalized | 0.342 | 0.680 | 0.592 | 0.573 | 0.411 | **0.477** |
| Trace++ (Self-Improve) | Normalized | 0.620 | 0.756 | 0.504 | 0.645 | 0.432 | **0.588** |

Table 4: **Theorem Quality Scores.** Average normalized scores in VeriBench split: Easy Set (Easy), CS Set (CS), Real Python Code (Real), HumanEval Set (HE), Security Set (Sec). All rows use Claude 3.7 as the agent model and as the LLM judge. Trace+ denotes the trace self-debug agent and Trace++ the self-debug and self-improve agent. The DSPy ReAct agent only performs self-debug. The LLM judge issues 5 scoring calls per file pair and returns an integer rubric score in $\{0, 1, \ldots, 10\}$. Normalized scores are computed by dividing by the maximum score (10) and averaging across all examples. The Avg Norm Score shows the overall average over the entire benchmark.

## 5 Discussion, Limitations, and Future Work

We study an LLM judge that *compares each candidate artifact against its task's gold reference* for Lean 4 code verification and a framework to increase confidence in it. The judge satisfies falsifiable score-level properties (identity and monotonicity) and, with Exp13, we additionally calibrate its binary decision boundary using full-dataset threshold sweeps with uncertainty estimates. Our data supports a clear separation: score ranking remains stable under threshold changes, while binary stability and coverage trade off in a tunable way. In experiments, certified judging produces informative method rankings and tracks engineering signals (Pass@$k$, unit-test accuracy); Easy and HumanEval show the strongest agreement, while Security and RealCode remain hardest [7].

**Limitations.** The judge is a *proxy*: scores are prompt/rubric-sensitive and do not constitute kernel-level proof checking; they certify agreement with a rigorous yet necessarily incomplete gold specification since no complete mechanical checks of specification equivalence exist [22]. Our certification relies on a finite perturbation suite, so untested failure modes may persist. Specifications that are exotic to this dataset may be correct but receive incorrect scores. Calibration improvements are modest in absolute magnitude and do not by themselves establish external validity. We now include two-pass human ratings and average-label analyses, but sample size remains limited ($n = 72$ aligned pairs and $n = 8$ judges), so some multivariate effects are underpowered. Future work will expand independent raters and problem coverage, strengthen semantic equivalence handling (candidate $\Rightarrow$ gold postcondition checks), add adversarial judge stress tests, and introduce standardized "judge cards" for reproducible, certified deployment [38].

## References


[1] * Ahuja et al. Improver: Lean-aware chain-of-states prompting for proof rewriting and repair. arXiv preprint, 2024. Add exact authors and arXiv ID if available.

[2] * Aniva et al. Pantograph: Programmatic access to lean 4 for learning-based proof search. arXiv preprint, 2025. Add exact authors and arXiv ID if available.

[3] Randy Bach et al. A systematic literature review of user trust in AI-enabled systems: An HCI perspective. *International Journal of Human–Computer Interaction*, 2022.

[4] Sergiu Bursuc, Theodore Ehrenborg, Shaowei Lin, Lacramioara Astefanoaei, Ionel Emilian Chiosa, Jure Kukovec, Alok Singh, Oliver Butterley, Adem Bizid, Quinn Dougherty, Miranda Zhao, Max Tan, and Max Tegmark. A benchmark for vericoding: formally verified program synthesis, 2025. URL `https://arxiv.org/abs/2509.22908`.

[5] Zheng Cheng Chan et al. Chateval: Towards better llm-based evaluators through multi-agent debate, 2024. URL `https://arxiv.org/abs/2308.07201`.

[6] Eric Chen, Aryan Gulati, Brando Miranda, Zeyu Tang, and Sanmi Koyejo. Rethinking LLM judges: Chain-of-thought and multi-step pipelines for math grading. In *ICLR 2026 Workshop*

*on Logical Reasoning of Large Language Models*, 2026. URL https://openreview.net/forum?id=vdXPorr099.

[7] Mark Chen et al. Evaluating large language models trained on code, 2021. URL https://arxiv.org/abs/2107.03374.

[8] Ching-An Cheng, Allen Nie, and Adith Swaminathan. Trace is the next autodiff: Generative optimization with rich feedback, execution traces, and llms, 2024. URL https://arxiv.org/abs/2406.16218.

[9] Z. Ding et al. Citations and trust in LLM-generated responses. In *AAAI*, 2025.

[10] * Kenton et al. Scalable oversight via debate and consultancy. arXiv preprint, 2024. Add exact venue/ID if available.

[11] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. Dspy: Compiling declarative language model calls into self-improving pipelines. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL https://openreview.net/forum?id=ICLR_2024_DSPy.

[12] Tom Kocmi et al. Llms instead of human judges? a large scale empirical study across 20 nlp evaluation tasks, 2024. URL https://arxiv.org/abs/2406.18403.

[13] Shulei Li et al. Can you trust llm judgments? reliability of llm-as-a-judge, 2024. URL https://arxiv.org/abs/2412.12509.

[14] * Liu et al. Safe: Verifying every intermediate step in lean 4 with the formalstep benchmark. arXiv preprint, 2025. Add exact authors and arXiv ID if available.

[15] Yang Liu et al. G-eval: Nlg evaluation using gpt-4 with better human alignment, 2023. URL https://arxiv.org/abs/2303.16634.

[16] W. Ma et al. Hypocompass: Using LLMs as teachable agents for debugging. In *International Conference on Artificial Intelligence in Education (AIED)*, 2024.

[17] Brando Miranda, Zhanke Zhou, Allen Nie, Elyas Obbad, Leni Aniva, Kai Fronsdal, Weston Kirk, Dilara Soylu, Andrea Yu, Ying Li, and Sanmi Koyejo. Veribench: End-to-end formal verification benchmark for ai code generation in lean 4. In *ICML 2025 Workshop on AI for Math (AI4Math)*, 2025. URL https://openreview.net/forum?id=rWkGFmnSNl. Poster.

[18] A. Naik et al. TiCoder: LLM-based test-driven interactive code generation. *IEEE Transactions on Software Engineering*, 2024.

[19] Fan Nie, Ken Ziyu Liu, Zihao Wang, Rui Sun, Wei Liu, Weijia Shi, Huaxiu Yao, Linjun Zhang, Andrew Y. Ng, James Zou, Sanmi Koyejo, Yejin Choi, Percy Liang, and Niklas Muennighoff. Uq: Assessing language models on unsolved questions, 2025. URL https://arxiv.org/abs/2508.17580.

[20] Fan Nie, Ken Ziyu Liu, Zihao Wang, Rui Sun, Wei Liu, Weijia Shi, Huaxiu Yao, Linjun Zhang, Andrew Y. Ng, James Zou, Sanmi Koyejo, Yejin Choi, Percy Liang, and Niklas Muennighoff. Uq: Assessing language models on unsolved questions, 2025. URL https://arxiv.org/abs/2508.17580.

[21] * Ospanov and * Yousefzadeh. Apollo: Recursive repair with lean feedback boosts minif2f proof success. arXiv preprint, 2023. Add exact arXiv ID if available.

[22] H. G. Rice. Classes of recursively enumerable sets and their decision problems. *Transactions of the American Mathematical Society*, 74(2):358–366, 1953. doi: 10.2307/1990888. URL https://doi.org/10.2307/1990888.

[23] * Robertson and Sanmi Koyejo. Step-by-step but verified: Incentive-compatible, robust queries without ground truth. arXiv preprint, 2025. Add exact arXiv ID if available.

[24] * Salaudeen et al. Validity-centered evaluation for llm judges: A measurement framework. arXiv preprint, 2025. Add exact authors and arXiv ID if available.

[25] Olawale Salaudeen and Moritz Hardt. Imagenot: A contrast with imagenet preserves model rankings, 2024. URL https://arxiv.org/abs/2404.02112.

[26] J. Shen et al. Symgen: Source-grounded, symbolically annotated generation for fast human verification. In *CLM Workshop*, 2024.

[27] * Sun et al. Clover: Closed-loop code–spec generation and smt-based verification. arXiv preprint, 2024. Add exact authors and arXiv ID if available.

[28] Yanfei Tong and Chiyuan Zhang. Codejudge: Evaluating code generation with large language models, 2024. URL https://arxiv.org/abs/2410.02184.

[29] VeriBench. Veribench: End-to-end formal verification benchmark for AI code generation in lean 4 (poster, ai4math @ icml 2025). https://icml.cc/virtual/2025/52376, 2025.
[30] * Wang et al. Theoremllama: Bootstrapping natural-language to lean 4 with iterative proof synthesis. arXiv preprint, 2024. Add exact authors and arXiv ID if available.
[31] S. Wang et al. Investigating and designing for trust in AI-powered code generation tools. In *ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, 2024.
[32] Tongshuang Wu et al. Promptchainer: Chaining LLM prompts through visual programming. In *CHI Late-Breaking Work*, 2022.
[33] J. Yan et al. Intelliexplain: Natural-language explanations for interactive code generation. In *Proceedings of the 2024 Conference on Human Factors in Computing Systems (CHI) Workshop/Under Review*, 2024.
[34] Lan Zhang, Marco Valentino, and Andre Freitas. Beyond gold standards: Epistemic ensemble of llm judges for formal mathematical reasoning, 2025. URL https://arxiv.org/abs/2506.10903.
[35] * Zheng et al. Scrit: Self-evolving critics for scalable llm oversight. arXiv preprint, 2024. Add exact authors and arXiv ID if available.
[36] Lianmin Zheng et al. Judging llm-as-a-judge with mt-bench and chatbot arena, 2023. URL https://arxiv.org/abs/2306.05685.
[37] * Zhou et al. Don't trust, verify: Autoformalizing math solutions in isabelle/hol improves robustness. arXiv preprint, 2024. Add exact authors and arXiv ID if available.
[38] Jin Peng Zhou, Sébastien M. R. Arnold, Nan Ding, Kilian Q. Weinberger, Nan Hua, and Fei Sha. Graders should cheat: privileged information enables expert-level automated evaluations, 2025. URL https://arxiv.org/abs/2502.10961.
[39] Tianyi Zhuo. Ice-score: Llm-as-a-judge for code without ground truth outputs. arXiv preprint, 2024. arXiv ID not specified in source; update if known.

# A Related Work (Cont.)

**Autoformalization.** Complementing this line of work, Wang et al. [30] present THEOREMLLAMA, a framework aimed at enhancing LLM translation into Lean 4. Drawing on over 100K proof examples from the Mathlib4 library, TheoremLlama employs a novel natural-language-to-formal-language (NL-FL) bootstrapping strategy and iterative proof synthesis. This enables the reuse of verified examples as templates for future translations. The framework achieves 36.48% and 33.61% accuracy on the MiniF2F-Valid and MiniF2F-Test benchmarks, respectively—surpassing GPT-4 by more than ten percentage points on both.

**Techniques for Code Verification.** IMPROVER [1] introduces a Lean-aware Chain-of-States prompting loop that integrates retrieval, best-of-$n$ sampling, and iterative correction to rewrite formal proofs with improved properties. By optimizing for metrics such as brevity and readability, ImProver reduces the number of tactics by half, doubles proof readability, and boosts theorem prover acceptance rates by over 80%. In addition, CLOVER [27] implements a closed-loop pipeline in which an LLM first generates code, docstrings, and formal annotations, then uses reconstruction-based prompting to enforce consistency across these outputs, and finally applies SMT-based verification to validate correctness. Evaluated on the CloverBench suite of Dafny programs, Clover accepts 87% of correct solutions, rejects 100% of flawed ones, and even uncovers bugs in human-written code—demonstrating the power of hybrid generation-verification pipelines. In a complementary direction, Zhou et al. [38] improve general-purpose LLM-based graders by augmenting them with "privileged" information such as gold-standard solutions, grading rubrics, and detailed annotations. When necessary, the system provides targeted hints back to candidate models. This approach achieves grading performance on par with or exceeding that of specialized systems—and even expert humans—on difficult programming benchmarks.

**Agentic Frameworks and Tools for Code Verification.** TRACE [8] proposes *generative optimization*, tuning entire computational workflows—including code, prompts, tool calls, and error signals—by treating execution traces as gradients in the OPTO framework. With a PyTorch-like API and the LLM-based optimizer OptoPrime, it supports diverse tasks such as prompt tuning, debugging, and robot control, rivaling specialized optimizers. Building on modular composition, DSPY [11] treats LLM calls as declarative modules in a computational graph. Users define concise input–output

signatures, and DSPy's compiler automatically bootstraps or fine-tunes pipelines using built-in "teleprompters." This enables a few-line programs to outperform expert-crafted prompts in math, QA, and agent workflows. Extending agentic capabilities to formal reasoning, PANTOGRAPH [2] offers a programmatic interface to Lean 4 with support for advanced proof search. It exposes internal proof states and tactics for integration with learning agents, replacing human-facing interfaces with API-level control.

### A.1 Related Work: Scalable Oversight

**Scalable Oversight with LLM Judges** Researchers have proposed multi-agent oversight frameworks such as debate and consultancy, where weaker LLMs act as judges supervising stronger agents [10, 5]. Debate, in which two models argue opposing sides before a judge, consistently improves truth-tracking compared to single-pass QA. Kenton et al. [10] demonstrate that debate reduces the likelihood of judges being misled in math and code tasks, highlighting it as a scalable oversight mechanism.

**Self-Improving Critics.** In the absence of stronger supervisors, models can bootstrap judgment skill through self-critique. The Self-Evolving Critic (SCRIT) framework trains LLMs to generate critiques of flawed outputs, validate corrections, and iteratively refine their ability to spot errors. Zheng et al. [35] report improvements of over 10% on critique benchmarks, showing that self-critique fosters scalable, autonomous oversight.

**Certifying Judges.** Naïve LLM judges are often inconsistent, rewarding stylistic cues over correctness [12, 13].

**Validity-Centered Evaluation.** Certifying judges requires not only behavioral properties but also principled validity evidence. Salaudeen et al. [24] introduce a framework that distinguishes between measurement, claims, and evidence, grounded in psychometrics. They emphasize construct, content, and criterion validity, and systematically enumerate threats to validity. This framework provides a blueprint for reporting and interpreting judge certification results, complementing identity/monotonicity checks with broader validity cards.

**Robust Evaluation without Ground Truth.** Standard judge queries can be brittle in the absence of ground truth, particularly under adversarial manipulation [39, 28]. Robertson and Koyejo [23] model the overseer as an agent and propose using information-theoretic, incentive-compatible queries instead of direct ratings. They introduce bounded $f$-divergence measures such as TVD-MI, which exhibit polynomial robustness to adversarial strategies, unlike unbounded KL-based criteria. This work highlights a pathway toward robust, certifiable judges when explicit ground truth is unavailable.

**Formal Methods as Oversight.** Proof assistants provide binary, kernel-level guarantees. Zhou et al. [37] autoformalize math solutions into Isabelle/HOL and show that filtering via proof checking outperforms majority voting by 12% on GSM8K. Liu et al. [14] extend this approach with SAFE, which verifies every intermediate step in Lean 4 and introduces the FormalStep benchmark, reducing hallucinations and enabling fine-grained oversight.

**Agentic Iterative Pipelines.** Ospanov and Yousefzadeh [21] integrate Lean feedback into recursive repair cycles, boosting general-purpose LLM proof success from 3–7% to over 40% on miniF2F, and achieving 75% with fewer than 1000 samples. Miranda et al. [17] extend this paradigm to program verification, showing that self-debugging agents quadruple success compared to one-shot prompting. Incorporating an LLM judge improved completeness on security tasks but required calibration to avoid regressions on simpler cases.

**Benchmarks for Judge Reliability.** Benchmarks such as VeriBench and FormalStep quantify judge reliability across compilation, proof success, and specification quality. Miranda et al. [17] report normalized specification scores of 0.470 for one-shot prompting versus 0.615 for self-debugging agents. Outside formal mathematics, MT-Bench and G-Eval provide complementary judge-evaluation settings using pairwise preferences and rubric-guided scoring, underscoring the need for calibration and validity reporting [36, 15]. Together with validity frameworks and robust information-theoretic scoring, these benchmarks advance the agenda of certifying LLM judges for formal verification.

**Evaluating Autoformalization with Reference-Free Judges.** In a complementary line of work, Zhang et al. [34] propose an ensemble of LLM judges for the autoformalization task, using a fine-grained taxonomy to score formalizations without a gold-standard reference. The primary strength of this approach is its reference-free nature, offering a scalable evaluation pathway when gold standards are unavailable or prohibitively expensive to create. However, the work's motivation for abandoning gold-standard comparison is based on limited evidence: a small, non-random sample of "potentially incorrect" formalizations pre-selected by an LLM, whose own reliability is uncertified. Furthermore, the framework validates its judge by correlating with human preference scores—a subjective proxy—rather than the more objective, machine-checkable goal of logical equivalence to a known-correct specification. Critically, even if minor errors exist in gold-standard datasets, recent work has shown that such noise rarely alters the relative ranking of models, as errors are typically correlated across systems [25]. This suggests that a small number of flawed examples does not invalidate the utility of an entire benchmark for comparative evaluation. Our work, in contrast, proceeds from the basis that gold standards are sufficiently reliable for ranking and focuses on certifying a judge's logical consistency against them – which is the desirable gold metric to be using.

### A.2 HCI for Solving Scalable Oversight

**From more evaluation to *human-centered* oversight.** HCI reframes scalable oversight as a *calibration* problem: design interactions that let people verify, steer, and place *appropriate* trust in LLM outputs rather than maximizing raw trust [3]. Interface choices (e.g., citations, explanations, confidence cues) can substantially shift user trust—sometimes in misleading ways—so the goal is to expose process and embed lightweight verification instead of presenting one-shot answers [9]. User studies in AI pair-programming similarly surface needs to communicate model performance, provide control over output style/constraints, and expose uncertainty to avoid over-reliance [31].

**Hybrid human↔AI evaluation.** HCI work operationalizes scalable evaluation by combining expert/lay judgments with automated judges. Humans annotate failures or verify slices of output; systems then train or condition LLM-based "reference evaluators" on those signals to scale up assessment while staying anchored to what users value (accuracy, safety, usefulness). This human-in-the-loop pattern complements LLM-as-judge approaches—whose reliability varies by task and setup [12, 13]—and is especially relevant when pass@k or single-shot metrics are brittle.

**Interfaces that *operationalize* oversight.** HCI systems instantiate oversight as interaction loops that make correctness checkable *during* generation: (i) *test-driven code generation* (TiCoder) elicits yes/no judgments about candidate unit tests before showing code; a few interactions reduced cognitive effort and lifted pass@1 by $\sim$45.7% on coding benchmarks [18]; (ii) *explain→correct loops* (IntelliExplain) pair each snippet with a natural-language rationale that users can critique, improving novice task success by $\sim$11–25% [33]; (iii) *pair debugging* (HypoCompass) has humans hypothesize faults while the LLM attempts fixes, yielding $\sim$12% learning gains [16]; (iv) *multi-step pipelines* (PromptChainer) expose intermediate steps (e.g., tests $\rightarrow$ verification $\rightarrow$ proof attempt), enabling user intervention where failures arise [32]; and (v) *source-grounded verification* (SymGen) highlights claim-level spans backed by exact sources and leaves unsupported text unhighlighted, speeding verification by $\sim$20% and raising user confidence [26]. Collectively, these patterns shift trust from fluency to *verifiable progress*.

**Applying HCI patterns to formal specs/theorems.** Formal kernels (Lean/Isabelle) already deliver binary checks; HCI contributes the *workflow glue* that helps users evaluate *which* theorem/spec to prove and whether it is complete. Two actionable patterns: (1) *counterfactual spec probes*—UI toggles that remove a postcondition or inject a known bug and display the impact on tests/judge score—help detect missing clauses or overfitting; and (2) *stepwise proof oversight*—surfacing intermediate goals and marking unverified suggestions—keeps the human in control of proof search. In our setting, these patterns interface naturally with a graded, semantics-sensitive evaluator.

### A.3 Vericode

Bursuc et al. [4] presumes well-formed formal specifications and evaluates end-to-end verification under those specs (in Dafny, Verus/Rust, or Lean) reporting verified pass rates. In practice, impactful

problems often lack both complete specifications and ground-truth answers. Our VERIBENCH program targets this gap by challenging the *generation* of specifications (from code, tests and documentation) and then *certifying* their quality prior to proof search using a Lean-aware LLM judge that we validate via falsifiable principles/properties (Identity, Monotonicity, and Prompt Stability/Invariance), addressing known pitfalls of LLM-as-a-judge such as bias and prompt sensitivity [36, 12, 13]. However, our framework has only been tested with a judge with gold references files with specifications but we will explore metrics/methods that don't require this like TVD-MI in [23].

Beyond spec-first settings, the Unsolved Questions (UQ) paradigm evaluates models on difficult, real-world problems that have neither ground-truth answers nor formal specifications; instead, it relies on validator pipelines and community verification to track progress without oracles [19, 20]. Taken together, these lines of work shift evaluation from "solve what is already specified" to "specify, certify, then solve": vericoding provides end-to-end rigor under given specs, VERIBENCH adds specification synthesis with a certified semantic judge, and UQ extends evaluation to specless, answerless settings that require validator-assisted screening. Our contribution is a certified, semantics-sensitive judge that enables scalable oversight when the specification must be proposed rather than merely proved. Because specification generation often lacks a unique ground truth, we will extend our oracle-free certification protocol with UQ-style validators, adding cycle-consistency tests (spec to code/docs) alongside our bug and ablation ladders, and enabling abstention when validators disagree to prioritize precision over coverage.

# B Discussion (Cont.)

**Why Lean 4?** Lean 4 is a full fledge programming language and lets VeriBench contain full runnable code and machine-checked proofs in the same dependently-typed language. Ahead-of-time compilation with the Lake toolchain produces fast native binaries, while first-class `Task` primitives, async I/O, and a thread-pool scheduler enable genuine concurrent programs inside the prover. Lean's C-level foreign-function interface (FFI) lets those binaries call out to high-performance libraries when needed. On the proof side, Lean ships a powerful metaprogramming system written in Lean itself, giving researchers a programming interface to access its internals. This led to the creation of tools such as Pantograph and Aesop. Finally, the community-maintained `mathlib4` and `std4` libraries supply thousands of reusable theorems and data structures, and they are expanding quickly thanks to an active Zulip and GitHub ecosystem. Lean also has a work-in-progress program verification library, CSLib, which has recently seen rapid growth. Lean 4 is backed by an unusually vibrant open-source community: hundreds of contributors refine `mathlib4` on GitHub each week, the public Zulip sees expert discussion around the clock, and even Fields-Medalist Terence Tao has chosen Lean to formalize portions of his current research—clear testimony to the ecosystem's accessibility and intellectual depth. Unlike the unit tests, fuzzers, and code sanitation pipelines common in industry—tools that sample inputs or rely on heuristics—Lean 4 supplies machine-checked proofs that a property holds for *all* executions. Its dependent type system can encode deep invariants (e.g., length-indexed arrays, bounded integers), so programs that violate them fail to compile, eliminating whole classes of bugs such as buffer overflows or integer wrap-around. Code, specification, and proof reside in the same file and are compiled by Lake into the shipped native binary, preventing the drift that arises when verification artifacts live outside the build. In short, Lean turns informal "best-effort" checks into formal, end-to-end guarantees without sacrificing performance or interoperability.

**Lean 4's limitations.** Lean's toolchain is still younger than Coq's or Isabelle's, making its standard libraries and automation smaller, and thus some formalizations demand extra groundwork. Lean's runtime has not been stress-tested at the scale of mainstream systems languages, meaning large-scale or safety-critical deployments may require additional vetting. Lean tactics could contain bugs and fail to typecheck on different environments. Acknowledging these gaps clarifies that VeriBench chooses Lean 4 for its unique unified programming-plus-proving model and modern automation hooks, not because it already matches the decades-old industrial maturity of older theorem provers. *Static vs. runtime caveat:* Lean's guarantees are *static* and specification-relative: they certify the Lean program meets the stated properties, but they do not by themselves detect emergent runtime faults outside the model (e.g., I/O (Input/Output) failures, environment misconfiguration, FFI (Foreign Function Interface) unsoundness, resource exhaustion, or undefined behavior in linked C code). Such behaviors must either be modeled and proved, or mitigated with production safeguards (monitoring, sandboxing, input validation).

**Lean 4 versus Dafny.** Unlike Dafny, whose verifier translates each program into the Boogie intermediate language and then discharges first-order verification conditions with an SMT solver such as Z3, Lean 4 reasons natively in a dependently-typed calculus. Because Lean 4's types can mention run-time data and the very same source code is ahead-of-time compiled to a native binary via Lake, we can both state and prove value-indexed, higher-order properties (e.g., length-indexed vectorsand run the verified program itself—an end-to-end, fidelity that Dafny's SMT-centred, Boogie-to-Z3 workflow cannot natively match. This gives Lean the expressive power to specify and prove higher-order, data-dependent properties—precisely the kind of semantic guarantees VeriBench seeks to test—while still yielding runnable binaries compiled by the same toolchain. Dafny's SMT-centric workflow offers impressive push-button automation for imperative code but cannot natively encode the richer specifications (e.g., length-indexed vectors, algebraic invariants) that Lean handles directly.

**Lean 4 versus TLA+.** TLA+ excels at high-level specification of concurrent and distributed protocols, with correctness checked by the TLC model-checker and the TLAPS proof system that dispatches first-order obligations to external provers. However, TLA+ specifications are not executable programs; a separate implementation step is required, and state-space explosion can limit model-checking scalability. VeriBench instead needs a prover where the specification, proof, and runnable code live in the same language. Lean 4's dependently-typed core lets us capture fine-grained, data-dependent invariants and then compile the very same artifacts to fast native binaries—capabilities outside TLA+'s scope.

**Lean 4 verus Verus.** Verus is a verification framework for Rust programs using SMT solvers. A user state intermediate goals and an SMT backend fills in the intermediate steps. Compared to Verus, users have the option to use or not use SMT Solver-based tactics in Lean. This enables more fine-grained control and steering of the proof process. Verus also has a much larger kernel compared to Lean, and is more susceptible to soundness problem than Lean whose kernel is small.

# C Future Work (cont.)

While [23] study demonstrates that total-variation-based mutual information (TVD-MI) provides robust, gaming-resistant evaluation without ground truth, many settings —such as VeriBench— do provide gold specifications in Lean 4. A promising direction is to develop hybrid protocols that combine TVD-MI's black-box robustness with supervised ground-truth checks (compile success, unit tests, post-condition proofs, and equivalence theorems). By calibrating MI scores against known gold outcomes, we can create composite metrics that preserve gaming-resistance while offering calibrated probabilities of correctness where supervision exists. This opens the door to semi-supervised label propagation, active selection of items to prove, and cross-system experiments in domains with heterogeneous supervision. Such an integration would extend Robertson's information-theoretic framework into a practical, specification-aware oversight system that leverages the best of both unsupervised robustness and supervised precision.

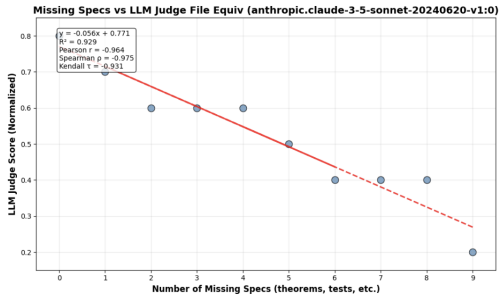
# D Why Judge Scores Are Specification-Relative

It is tempting to interpret a perfect score from our LLM judge as a guarantee of absolute program correctness. However, this is fundamentally impossible due to established limits from computability theory. The obstacle is not a lack of ingenuity in our benchmark design or judge prompting, but an established negative result.

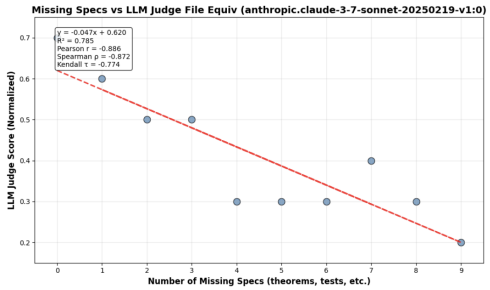
**Rice's Theorem.** The core limitation stems from Rice's theorem, which states that any non-trivial, semantic property of programs in a Turing-complete language is undecidable.

> **Rice's Theorem (Informal):** Let $\mathcal{S}$ be any property of a program's input-output behavior (e.g., "the program always terminates" or "the output is always sorted"). If $\mathcal{S}$ is non-trivial (meaning some programs have it and some don't), then no algorithm can exist that decides for *every* program whether it satisfies $\mathcal{S}$.

**Impact on the Gold Standard and the Judge.** Rice's theorem directly implies that the gold reference files used by our judge are necessarily incomplete. If a recursively enumerable gold standard could capture *every* true semantic property of a program, one could use it to build a decision




(a) Claude 3.5 Sonnet v1: strong monotonic decrease ($y = -0.056x + 0.771$, $R^2 = 0.929$, Pearson $r = -0.964$, Spearman $\rho = -0.975$).




(b) Claude 3.7 Sonnet v1: clear negative trend with minor non-monotonicity ($y = -0.047x + 0.620$, $R^2 = 0.785$, Pearson $r = -0.886$, Spearman $\rho = -0.872$).

Figure 5: **(P3) Monotonicity w.r.t. missing specifications.** Candidate scores decrease as more theorems/tests are withheld. We retain both model variants to show robustness of the trend; 3.7's small non-monotonicity motivated the simplified rubric used in the main results.

procedure that contradicts the theorem. Therefore, our gold standards, while rigorous, represent a curated but fundamentally finite set of important specifications.

This leads to a critical conclusion for our work: the judge's evaluation is **specification-relative**. A perfect score does not certify universal semantic correctness; rather, it certifies perfect **conformance to our specific gold standard**. Acknowledging this limitation is essential for correctly interpreting the judge's scores and understanding the scope of our evaluation.

# E Judge Certification: Additional Plots

We report raw scatterplots used during certification of the LLM judge. These plots visualize the judge's response to controlled perturbations: (P1) identity/reflexivity, (P2) cumulative bug injection, and (P3) removal of specification items (tests/theorems). Figures A.1–A.3 show that scores drop as artifacts worsen; the bug plot is noisier with an early prompt, which motivated the simplified rubric used in the main experiments. P4 (prompt stability/invariance) is reported in the main text with dedicated aggregate plots.

*Protocol variants.* The scatterplots below are "sanity-check" runs with the simplest prompts to stress visible monotonic behavior. In the bug study (Fig. 6) the earlier, theorem-equivalence prompt produced a noisier trend; this prompted a simplified, rubric-first prompt that we use for all reported results in the main paper, yielding the stronger correlations cited in Fig. 1. For missing-specs (Fig. 5), both Claude 3.5 and 3.7 show clear negative trends; minor non-monotonicity in 3.7 disappears with the finalized rubric.

We note that, for the experiment shown in Figure 5, a more complex judge (Claude 3.7) does not satisfy the property that the LLM Judge Score should monotonically increase with the number of missing specs. Indeed, the Judge File Equivalence score was not monotonic, even for Claude 3.5, leading us to simplify the rubric used in our main paper.

# F Final Robust Human-Alignment Appendix

This appendix reports full visual diagnostics for Exp16/Exp17 under final robust settings (three human label sets: pass1, pass2, avg).

## F.1 Naive Codex-Style Blind Baseline

To contextualize certified judge performance, we evaluate a naive blind baseline on the same pair set ($n = 72$ overlap), using a single-pass, surface-form-oriented Codex-style rubric without certification constraints (no monotonicity/stability calibration). The baseline tracks pass2 labels somewhat but

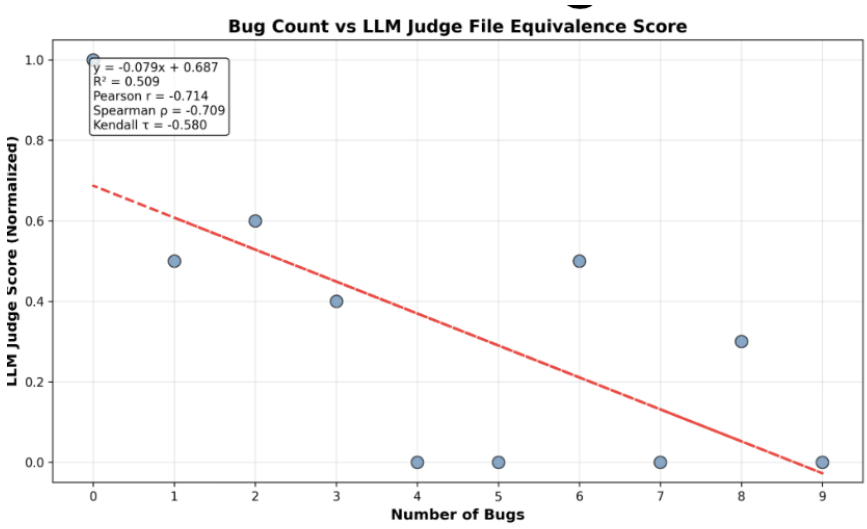
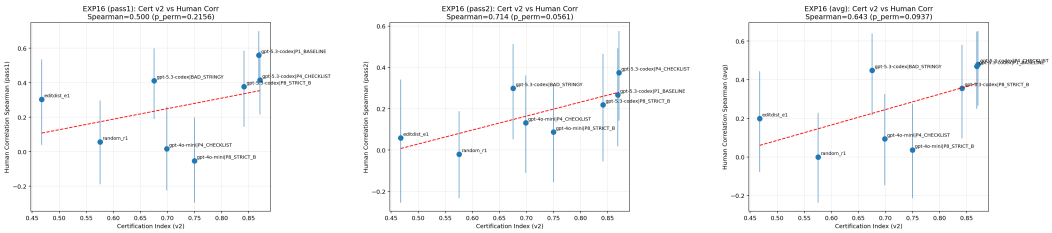



Figure 6: **(P2) Non-Monotonicity w.r.t. cumulative bugs (Claude 3.5 Sonnet).** Scores drop as more semantic bugs are injected (Pearson $r = -0.714$, Spearman $\rho = -0.709$, $R^2 = 0.509$); the higher noise with the initial "theorem-equivalence" prompt led us to simplify the rubric used for the final judge in the main paper.



(a) Exp16 trend (pass1) (b) Exp16 trend (pass2) (c) Exp16 trend (avg)

Figure 7: **Judge-level certification-to-human trend across all human label sets (Exp16).** The strongest and most stable effect is for trend-v1 certification, with averaged labels giving the largest effect size.

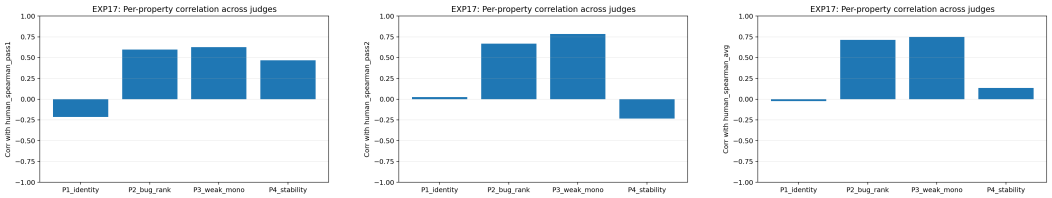
remains substantially weaker and less consistent than certified judges, especially on pass1 and in absolute error.

### F.2 Human-Human Agreement Ceiling Context

Judge-human correlations should be interpreted relative to the reliability of human labels themselves. Across Ethan pass1 vs pass2 on the aligned set ($n = 72$), inter-pass agreement is moderate (Spearman $0.249$, Pearson $0.375$, Kendall $0.221$, QWK $0.372$). Thus, judge-human correlations in the $0.48$–$0.56$ range are non-trivial and should be interpreted as approaching the available annotation ceiling in this dataset rather than as "low absolute" values.

### F.3 Cross-Model Validation Status (Full vs Preliminary)

To avoid over-claiming, we separate full-scale and preliminary evidence. Main conclusions in this paper use full-scale runs only (Exp16/Exp17: $n = 8$ judges, $n = 72$ aligned rows, pass1/pass2/avg label sets). Claude Sonnet certification is currently preliminary (3 files) and is included as directional evidence only, not as a core statistical pillar.




(a) Exp17 per-property (pass1) (b) Exp17 per-property (pass2) (c) Exp17 per-property (avg)

Figure 8: **Property-wise human alignment diagnostics (Exp17).** P3 weak monotonicity is consistently the strongest predictor across label sets; P2 is second strongest; P1/P4 are weaker as standalone predictors at this scale.

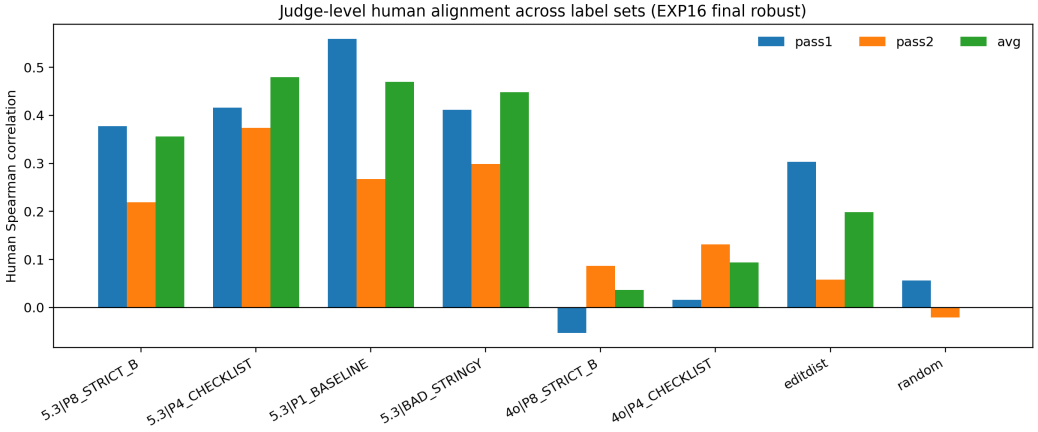



Figure 9: **Unplotted diagnostic added for this version:** judge-wise human Spearman across pass1/pass2/avg. GPT-5.3-Codex variants remain above weaker baselines on most label sets; pass-to-pass variation is visible and motivates reporting all three label sets.

### F.4 Failure Modes Found by Certification (Not Hidden)

The certification framework is useful precisely because it identifies failures: **(i) alpha-renaming sensitivity** is the dominant identity/invariance gap, and **(ii) local bug-ladder reversals** occur despite overall negative monotone trends. We therefore report both strengths and weaknesses, and include alpha-ablation diagnostics. On trend-v1 Spearman (certification $\rightarrow$ human), excluding alpha variants changes pass1 $0.810 \rightarrow 0.690$, pass2 $0.810 \rightarrow 0.714$, avg $0.881 \rightarrow 0.786$, showing that alpha cases materially affect measured robustness.

### F.5 Newest Results (Latest Rerun at Appendix Bottom)

To reduce judge-call noise, we reran Exp16 with higher repeats and robust repeat aggregation: `JUDGE_REPEATS=9 REPEAT_AGG=median BOOTSTRAP_ITERS=10000 PERM_ITERS=20000`. This run writes to `results_exp16_final_robust_r9_median`.

We also generated a low-variance replot package from the same detail rows (`results_exp19_low_variance_replot`): a cleaner main monotonicity curve for core judges plus separate failure-mode plots for alpha sensitivity and local bug reversals. Reversal rates in this package are: GPT-5.3 P4/P1 $= 0.00$, GPT-5.3 P8 $= 0.167$, BAD_STRINGY $= 0.833$, GPT-4o variants $= 0.667$–$0.750$, edit-distance/random $= 1.00$.

| Label Set | $\rho_s$ | $r$ | $\tau$ | MAE |
|---|---|---|---|---|
| pass1 | 0.171 ($p = 0.151$) | 0.218 ($p = 0.0656$) | 0.155 ($p = 0.144$) | 1.049 |
| pass2 | 0.388 ($p = 7.58 \times 10^{-4}$) | 0.427 ($p = 1.87 \times 10^{-4}$) | 0.343 ($p = 9.40 \times 10^{-4}$) | 1.333 |

Table 5: **Naive blind baseline vs. human labels (0–5 scale).** $\rho_s$: Spearman, $r$: Pearson, $\tau$: Kendall; MAE is absolute error on the 0–5 scale. This baseline is included as a sanity-check negative control and is not used for final ranking claims.

| Quantity | Value | Notes |
|---|---|---|
| Human-human Spearman (pass1 vs pass2) | 0.249 | 95% CI = $[-0.015, 0.489]$ |
| Human-human Pearson (pass1 vs pass2) | 0.375 | 95% CI = $[0.076, 0.604]$ |
| Human-human QWK (pass1 vs pass2) | 0.372 | 95% CI = $[0.073, 0.581]$ |
| Best judge-human Spearman (avg labels) | 0.480 | GPT-5.3-Codex P4_CHECKLIST |
| Best judge-human Spearman (pass1 labels) | 0.559 | GPT-5.3-Codex P1_BASELINE |

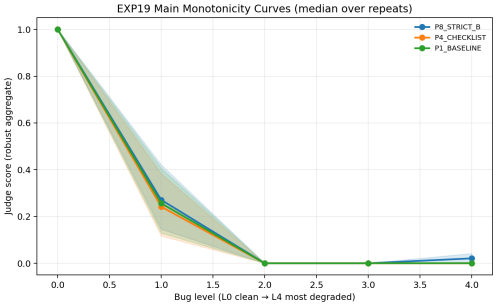
Table 6: **Human-human agreement ceiling context.** Inter-pass human agreement is moderate, so judge-human values near 0.5 are meaningful on this benchmark and should be interpreted with label-noise context.

| Judge Setup | Scale | In Main Claims | Role |
|---|---|---|---|
| GPT-5.3-Codex prompts (P8/P4/P1) | Full VeriBench pairs | Yes | Primary certified judges |
| GPT-5.3-Codex BAD_STRINGY | Full VeriBench pairs | Yes | Same-model stress baseline |
| GPT-4o-mini prompts (P8/P4) | Full VeriBench pairs | Yes | Cross-family baseline |
| Edit-distance / random baselines | Full VeriBench pairs | Yes | Non-LLM controls |
| Claude Sonnet certification run | 3 files (prelim) | No | Directional cross-model check |

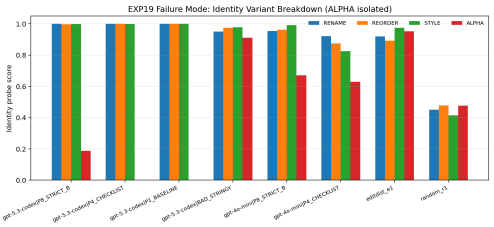
Table 7: **Cross-model validation status.** This paper's inferential claims are grounded in full-scale runs; Claude results are explicitly preliminary and not used as primary evidence.

| Label Set | v1 Spearman | v2 Spearman |
|---|---|---|
| pass1 | 0.762 ($p = 0.0372$, CI $[-0.139, 1.000]$) | 0.524 ($p = 0.1959$, CI $[-0.400, 0.975]$) |
| pass2 | 0.810 ($p = 0.0222$, CI $[0.291, 1.000]$) | 0.571 ($p = 0.1477$, CI $[-0.231, 0.975]$) |
| avg | 0.762 ($p = 0.0402$, CI $[0.103, 1.000]$) | 0.476 ($p = 0.2426$, CI $[-0.425, 1.000]$) |

Table 8: **Latest Exp16 rerun (R9 + median aggregation).** Increasing repeats and using median aggregation preserves the v1 trend signal while v2 remains more sensitive at this $n_{\text{judges}} = 8$ scale.




(a) Main low-variance monotonicity curves (core GPT-5.3 judges, robust repeat aggregation).




(b) Failure-mode isolation: identity variant breakdown with ALPHA shown separately.

Figure 10: **EXP19 replot package: cleaner main trend + explicit failure separation.** The left panel is the "main story" plot with lower visual variance; the right panel isolates alpha-renaming sensitivity rather than mixing it into the primary monotonicity figure.

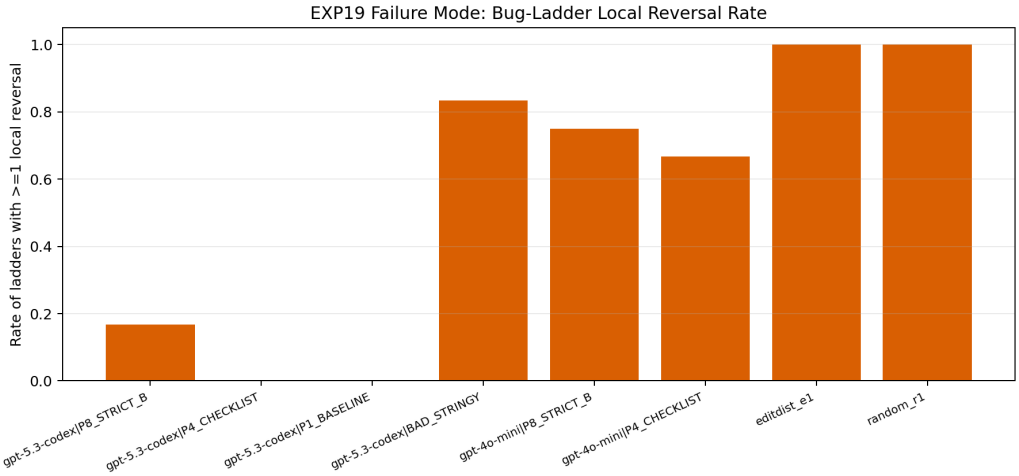



Figure 11: **EXP19 failure-mode plot: local bug-ladder reversal rates by judge.** This explicitly surfaces where monotonicity fails locally, separating robustness diagnostics from the primary trend plot.